



**SHAHEED HIMAYUN MUZZAMIL MEMORIAL
GOVERNMENT DEGREE COLLEGE ANANTNAG**

Department of Computer Applications

**Project Report (e-yantra)
Object Tracking Robot**

Batch 2022 (NEP)

Aqib Javid Bhat	65274-ANG-2022
Mohd Numan Asgar	65301-ANG-2022
Danish Ellahi	65277-ANG-2022
Ubaid Mushtaq Mir	65291-ANG-2022
Wasiq Tariq	63848-ANG-2021
Ifham Mushtaq	65292-ANG-2022

**Under the Supervision of
Dr. Tawseef Ahmed Teli
Asst. Prof. Department of Computer Applications**

Contents

Abstract	2
1. Introduction.....	2
1.1 Background and Motivation.....	2
1.2 Problem Statement	2
1.3 Scope and Applications.....	2
2. Methodology.....	3
2.1 Working Principle	3
2.2 Hardware Components Used.....	3
2.2.1 Arduino UNO	3
2.2.2 IR Sensor	4
2.2.3 Ultrasonic Sensor	4
2.2.4 L298N Motor Driver.....	5
2.2.5 Dual Shaft BO Motors	5
2.2.6 65mm BO Motor Wheels	5
2.2.7 Trolley Wheel / Caster Wheel.....	6
2.2.8 2WD Chassis.....	6
2.2.9 170-Ponit Mini Breadboard.....	7
2.2.10 Jumper Wires	7
2.2.11 18650 Lithium-Ion Rechargeable Batteries.....	7
2.2.12 18650X2 Battery Holder.....	8
2.2.13 ON/OFF Switch	8
2.3 Software and Algorithms Used.....	8
2.3.1 Programming Language and Platform Used.....	8
2.3.2 Libraries Used.....	8
2.3.3 Algorithm and Pseudocode	9
2.3.4 Flowchart.....	11
3. Implementation.....	13
3.1 Build and Assembly	13
3.2 Challenges faced and how they were resolved.....	14
4. Testing and Results.....	15
4.1 Tests Performed.....	15
4.2 Observations and Outcomes	15
5. Conclusion and Future Scope.....	16
6. GitHub Repository.....	16

Abstract

The development of autonomous robots capable of tracking objects in dynamic environments poses significant challenges, particularly in maintaining consistent proximity to targets without collisions or disengagement. This project presents an Object Tracking Robot that utilizes threshold-based control, integrating an IR sensor for object detection and an ultrasonic sensor for distance measurement to enable continuous tracking. Built on a custom Arduino Uno-based platform with a two-wheel drive system, the robot adjusts its behaviour based on predefined distance thresholds: moving forward within 8-17 cm, reversing when closer than 7 cm, and stopping if the object is beyond 17 cm. The design employs uniform motion for straight-line tracking and reverse movement for safety, ensuring reliable performance. Testing demonstrated stable tracking with precise distance maintenance, validated through experimental calibration. This cost-effective prototype integrates sensor feedback and real-time decision-making, serving as a foundation for interactive robotics applications. Future enhancements may include proportional control, additional sensors, or vision systems for advanced tracking. Potential applications include surveillance, educational tools, and assistive devices.

1. Introduction

1.1 Background and Motivation

Robotics has increasingly demanded systems capable of interacting with their environments beyond simple avoidance. Basic navigation approaches often fail to sustain engagement with objects, leading to disconnections or inefficient pauses in scenarios requiring ongoing proximity, such as monitoring or guidance tasks. This limitation inspired the development of an obstacle-tracking system, a key aspect of interactive robotics that leverages sensor data for reliable target following. Drawing from real-world examples like assistive robots that track users or inspection devices that maintain alignment with assets, this project uses an IR sensor for detection and an ultrasonic sensor for precise ranging. The aim is to create a robust framework that minimizes distance deviations and enhances functionality in dynamic settings.

1.2 Problem Statement

The central problem is to develop an autonomous mobile robot that can reliably detect an object in its immediate vicinity and maintain a specified following distance. The robot must be able to move forward to track the object, stop when the object is out of range, and reverse to avoid a collision if the object gets too close. The technical challenge lies in integrating an IR sensor for initial object detection with an ultrasonic sensor for accurate ranging. A key issue to overcome is the inherent limitation of IR sensors, whose performance is highly dependent on the target object's colour, material, and reflectivity, as well as ambient lighting conditions. The system must be calibrated to function effectively under these variable conditions, executing smooth transitions between forward, reverse, and stop states based on real-time sensor feedback.

1.3 Scope and Applications

The scope of this project is to design and build a custom two-wheel-drive (2WD) robot from scratch. The robot's intelligence is driven by an Arduino Uno, which processes inputs from a front-mounted IR sensor and an ultrasonic sensor. The control algorithm is based on a set of

predefined distance thresholds (TOO_CLOSE, FOLLOW_DISTANCE) to trigger forward, reverse, or stop commands. The project focuses on single object tracking in a forward-facing direction on a flat, indoor surface. It does not include advanced functionalities like object recognition, lateral tracking, or dynamic obstacle avoidance beyond the primary target.

This simple yet effective object-tracking robot has numerous real-world applications. In logistics and warehousing, similar robots can be used as "smart carts" that follow workers through aisles to assist in order picking, reducing physical strain and improving efficiency. In personal and assistive robotics, this technology can be integrated into smart luggage that follows its owner through an airport or shopping carts that trail a customer. For the elderly or visually impaired, it could power an assistive device that follows a caregiver. In security and surveillance, a fleet of such robots could be programmed to follow and monitor suspicious individuals or objects within a restricted area. Finally, it serves as an excellent educational tool for teaching fundamental concepts in electronics, programming, and robotics engineering.

2. Methodology

2.1 Working Principle

The Object Tracking Robot operates on a reactive feedback loop, using a front-mounted IR sensor to detect objects and a co-aligned ultrasonic sensor to measure distance, guiding its actions. Upon initialization, the Arduino configures pins and sets thresholds: TOO_CLOSE at 7 cm for reversal and FOLLOW_DISTANCE at 17 cm for forward movement. In the main loop, the IR sensor provides a digital output (LOW indicates detection within 2-20 cm). If an object is detected, the ultrasonic sensor measures the distance in centimetres. The logic then determines the action: reverse at 100 speed if ≤ 7 cm to create space, move forward at 120 speed if ≤ 17 cm to maintain or close the gap, or stop if beyond 17 cm to await re-detection. If no object is detected (HIGH), the robot stops immediately. This cycle runs continuously, with Serial monitoring for debugging. The two-wheel system, controlled via an L298N driver, uses uniform motion with PWM for speed adjustments. Safety is ensured through calibrated thresholds that balance pursuit and avoidance, preventing oscillations, and the design prioritizes engagement over evasion, suitable for indoor tracking without complex computations.

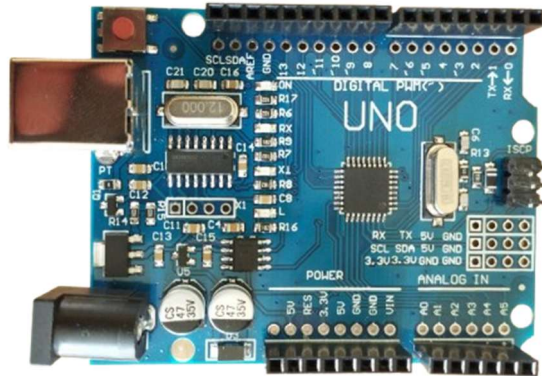
2.2 Hardware Components Used

The hardware for this project utilized standard components assembled into a custom design on a 2WD chassis, ensuring precise front-sensor alignment for tracking without pre-built platforms, emphasizing modular construction for adaptability. This section details the key electronic and mechanical elements.

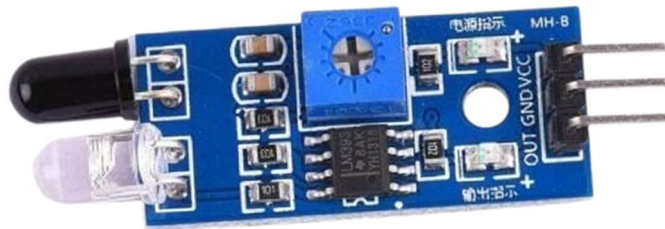
2.2.1 Arduino UNO

The Arduino Uno is a microcontroller board based on the ATmega328P. It features 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz quartz crystal, USB connection, power jack, ICSP header, and a reset button. It operates at 5V and can be powered via USB or external power supply.

In this project, it serves as the central controller, reading IR and ultrasonic data, executing threshold logic, and sending PWM/digital signals to the motor driver for real-time tracking responses.

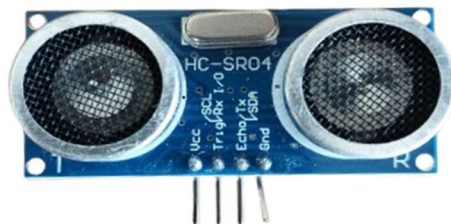


2.2.2 IR Sensor



The IR sensor emits infrared light and detects reflections to indicate proximity, operating at 3.3-5V DC with a digital output (LOW when an object is within 20 cm). Sensitivity is adjustable via a potentiometer. Mounted at the front, it triggers object detection, interfacing via a single digital pin to enable efficient tracking initiation.

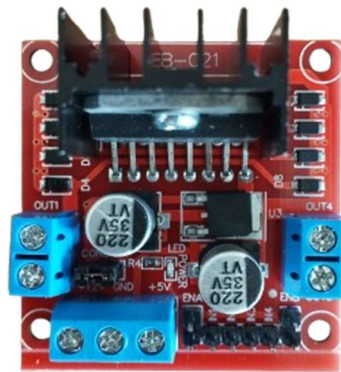
2.2.3 Ultrasonic Sensor



The HC-SR04 ultrasonic sensor consists of a transmitter and a receiver. It operates on a 5V DC supply and measures distance using sound waves at 40kHz. It has a detection range from 2 cm to 500 cm with an accuracy of about 3 mm. The sensor sends out an ultrasonic pulse and measures the time taken for the echo to return.

Aligned with the IR sensor at the front, it provides precise distance data via trigger and echo pins connected to the Arduino, supporting threshold decisions.

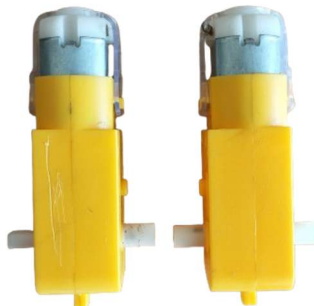
2.2.4 L298N Motor Driver



The L298N motor driver module is a dual H-bridge driver that can control two DC motors independently. It can handle up to 2A of current per channel and supports voltage inputs from 5V to 12V. It has onboard terminal blocks for motors and power, and input pins for control signals from the Arduino.

It connects low-power Arduino signals to the motors, enabling forward and reverse movements through direction pins and PWM for speed control.

2.2.5 Dual Shaft BO Motors



Each BO motor operates on 3V–12V DC and provides moderate torque with low RPM, typically around 100 RPM at 6V. The dual shaft allows for attachments on both ends, useful for future upgrades like encoders for speed sensing.

These motors drive the robot's wheels and are responsible for its mobility. Their torque is suitable for lightweight robotic platforms, allowing smooth motion over flat indoor surfaces. The dual-shaft design also offers flexibility for expansion.

2.2.6 65mm BO Motor Wheels

These wheels have a diameter of 65mm and are made of lightweight plastic with a rubber outer ring for better grip. They are designed to be press-fitted onto BO motor shafts.

The wheels enable the robot to move forward, backward, and turn efficiently. Their size provides a good balance between speed and stability, making them suitable for indoor obstacle-avoiding applications.



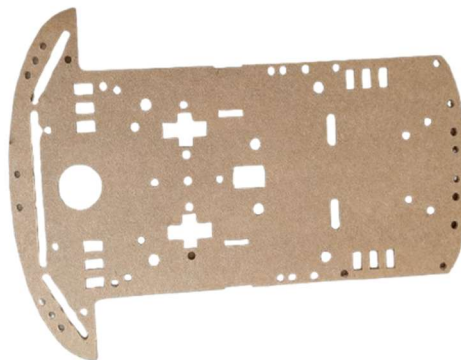
2.2.7 Trolley Wheel / Caster Wheel



The caster wheel is a freely rotating wheel that can swivel in 360 degrees. It is small in size, usually made of plastic or metal, and is mounted on a metal bracket.

This wheel supports the front end of the robot, maintaining balance while allowing free movement in all directions. It complements the two driven wheels, enabling smooth turns and preventing the robot from tipping.

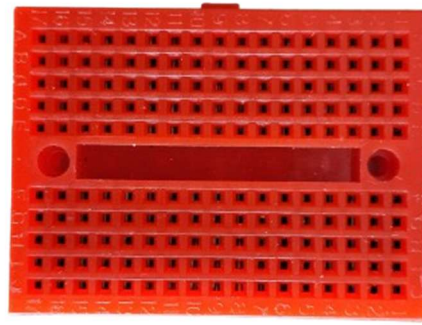
2.2.8 2WD Chassis



The 2-wheel drive chassis is typically made from acrylic or plastic and includes mounting holes for motors, battery holders, and control boards. It is lightweight, compact, and designed for BO motor compatibility.

The chassis provides the structural base for mounting all the components. It organizes the robot into a stable form factor and ensures that everything stays securely in place during movement and testing.

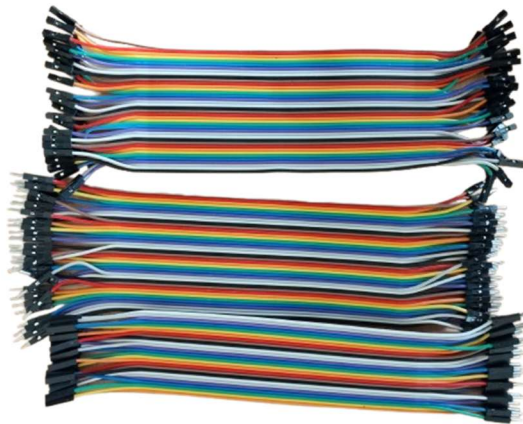
2.2.9 170-Pin Mini Breadboard



This is a compact solderless breadboard with 170 tie-points. It is made of ABS plastic and features two power rails along with connecting strips for temporary circuit construction.

The breadboard is used to connect components like sensors, modules, and wires during the prototyping phase. It helps in testing and modifying the circuit quickly without soldering, which is ideal for student projects.

2.2.10 Jumper Wires



Jumper wires are insulated electrical wires with connector pins at each end. They come in various types such as male-to-male, male-to-female, and female-to-female, and are typically used on breadboards and headers. These wires are available in different lengths and colors for easy circuit tracing.

Jumper wires are used for making electrical connections between the Arduino, sensors, LCD, motor driver, and breadboard. They enable flexible and temporary wiring, allowing quick prototyping and easy troubleshooting without the need for soldering. They also help in organizing the connections and keeping the circuit modular.

2.2.11 18650 Lithium-Ion Rechargeable Batteries



In this project, Hongli 2000mAh 3.7V 18650 Li-Ion Cell Batteries are used as high-capacity rechargeable power sources, providing approximately 3.7V each with a 2000mAh rating for sustained runtime. Two such batteries, connected in series via the 18650X2 holder, deliver around

7.4V to power the L298N motor driver, which in turn supplies the Arduino, motors, and other circuits of the robot.

2.2.12 18650X2 Battery Holder

The 18650X2 battery holder accommodates two 18650 lithium-ion batteries. It supplies power to the motors and the Arduino via the motor driver.



2.2.13 ON/OFF Switch



The On/Off switch is a simple SPST (Single Pole Single Throw) toggle switch. It usually has two terminals and is designed to interrupt or complete a circuit, allowing current to flow only when in the “ON” position.

The switch is used to conveniently power the robot on or off without disconnecting the batteries. It adds a layer of safety and ease of use by allowing quick shutdowns during testing or in case of malfunction and helps conserve battery life when the robot is idle.

2.3 Software and Algorithms Used

This section outlines the programming tools, logic, and flow that control the behavior of the robot.

2.3.1 Programming Language and Platform Used

The entire robot is programmed using the Arduino IDE, which is a user-friendly open-source platform for writing and uploading code to the Arduino Uno. The language used is C/C++, specifically tailored for embedded systems programming. The Arduino IDE offers built-in functions for controlling digital and analog pins, reading sensor data, and communicating with external devices, which made it easy to implement the logic required for this project.

2.3.2 Libraries Used

Libraries in Arduino are pre-written code that simplify programming by allowing users to interact

with hardware using simple functions instead of dealing with complex code. To help build the robot and make the development process smoother, the NewPing.h library was used.

The *NewPing.h* library is specifically used for working with ultrasonic sensors, which are used to measure distance. It makes controlling ultrasonic sensors much easier by providing easy-to-use functions for triggering and receiving echoes. In this project, it was used to manage the front ultrasonic sensor, enabling reliable and efficient distance measurement. This reduced code complexity and improved sensor response time, ensuring smoother real-time control.

2.3.3 Algorithm and Pseudocode

The algorithm for the Object-Tracking Robot uses an IR presence sensor to decide when a target is present and an ultrasonic sensor to measure its distance. The robot maintains a simple distance-based policy: reverse if too close, move forward if within the follow band, and stop if the object is too far or not detected. Parameters such as distance thresholds (TOO_CLOSE, FOLLOW_DISTANCE), PWM speeds (forward/reverse), and sensor sanity checks are platform and environment-dependent. Tune them incrementally based on motor characteristics, robot mass, wheel diameter, and ultrasonic reliability (surface reflectivity, ambient noise).

2.3.3.1 Algorithm

Objective: Implement autonomous object tracking using IR and ultrasonic sensors to maintain a target following distance from an object ahead.

1. Initialization Phase
 - i. Configure motor control pins (PWM and direction pins for left and right motors)
 - ii. Configure IR sensor pin for object detection
 - iii. Configure ultrasonic sensor pins (trigger and echo) for distance measurement
 - iv. Define control parameters: TOO_CLOSE (7 cm), FOLLOW_DISTANCE (17 cm), MAX_DISTANCE (500 cm), MOTOR_SPEED (100)
 - v. Display initial status on serial monitor
2. Sensor Reading Phase
 - i. Continuously read IR sensor for object presence
 - ii. Continuously read ultrasonic sensor for object distance
 - iii. Store readings in respective variables
 - iv. Output sensor readings to serial monitor for debugging
3. Object Detection and Tracking Phase

If IR sensor detects object (LOW) and distance is valid (≥ 0):

 - If distance $\leq$ TOO_CLOSE:
 - Reverse to avoid collision
 - Print "REVERSE" status
 - Else if distance $\leq$ FOLLOW_DISTANCE:
 - Move forward to maintain following distance
 - Print "FORWARD" status
 - Else
 - Stop motors (object too far)
 - Print "STOP" status

If IR sensor does not detect object (HIGH) or distance invalid:

 - Stop motors
 - Print "STOP" status

4. Loop Continuation

Repeat sensing and control phases continuously for real-time tracking.

2.3.3.2 Pseudocode

BEGIN Object Tracking Robot

 Initialize()

 Configure motor pins (ENA, IN1, IN2, ENB, IN3, IN4)

 Configure IR sensor pin (IR_FRONT)

 Configure ultrasonic pins (TRIG_PIN, ECHO_PIN)

 Set TOO_CLOSE $\leftarrow$ 7 cm

 Set FOLLOW_DISTANCE $\leftarrow$ 17 cm

 Set MAX_DISTANCE $\leftarrow$ 500 cm

 Set MOTOR_SPEED $\leftarrow$ 100

 Print initial status to serial

END

WHILE TRUE

 // READ SENSORS

 irFront $\leftarrow$ read IR sensor

 distance $\leftarrow$ read ultrasonic sensor

 Print irFront, distance to serial

 // OBJECT TRACKING LOGIC

 IF irFront = LOW AND distance $\geq$ 0 THEN

 IF distance $\leq$ TOO_CLOSE THEN

 Reverse motors at MOTOR_SPEED

 Print "REVERSE" to serial

 ELSE IF distance $\leq$ FOLLOW_DISTANCE THEN

 Forward motors at MOTOR_SPEED + 20

 Print "FORWARD" to serial

 ELSE

 Stop motors

 Print "STOP" to serial

 END IF

```

ELSE

    Stop motors

    Print "STOP" to serial

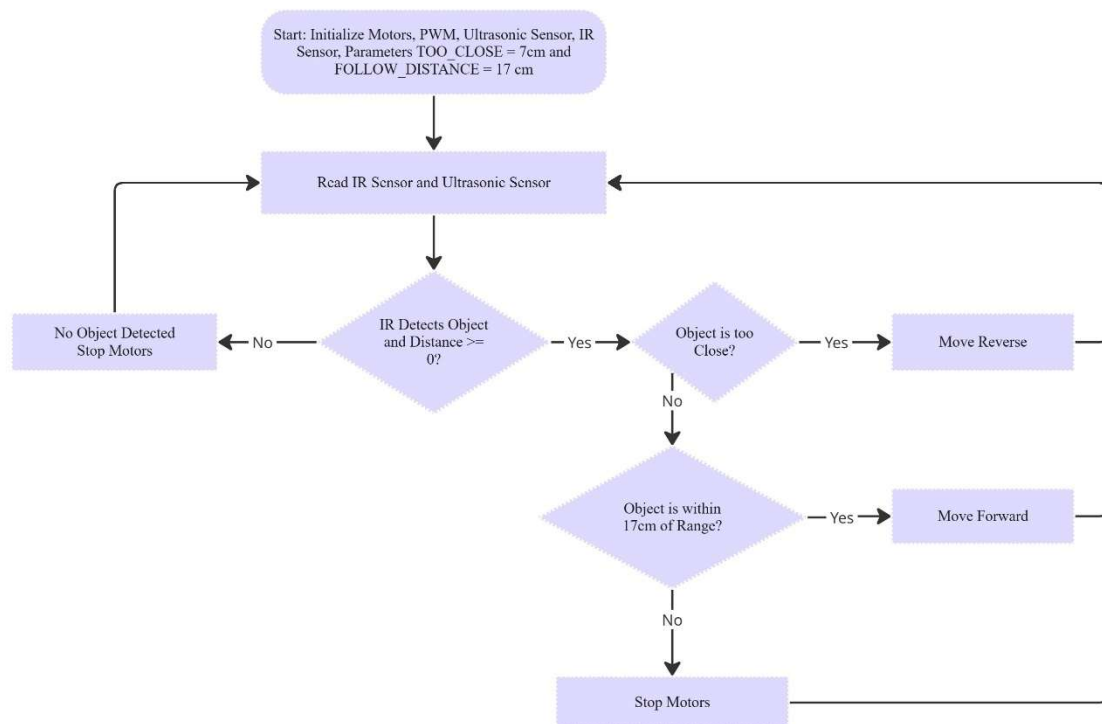
END IF

END WHILE

END Object Tracking Robot

```

2.3.4 Flowchart



2.3.5 Code

```

/*
  ObjectTrackingRobot.ino
  An autonomous object tracking robot using IR and ultrasonic sensors for real-
  time following. The robot detects nearby objects with an infrared sensor and
  maintains a safe following distance using ultrasonic distance measurement,
  automatically reversing when too close and stopping when the object is out of
  range.
*/
#include <NewPing.h>

// Pin Definitions
#define IR_FRONT 2 // Front IR sensor
#define TRIG_PIN 3 // Ultrasonic trigger
#define ECHO_PIN 4 // Ultrasonic echo

```

```
// Motor Driver Pins (L298N)
#define ENA 9    // Right motor speed (PWM)
#define IN1 8    // Right motor direction
#define IN2 7    // Right motor direction
#define ENB 10   // Left motor speed (PWM)
#define IN3 6    // Left motor direction
#define IN4 5    // Left motor direction

// Distance Settings
#define TOO_CLOSE 7
#define FOLLOW_DISTANCE 17
#define MAX_DISTANCE 500

// Speed Settings
#define MOTOR_SPEED 100

NewPing sonarFront(TRIG_PIN, ECHO_PIN, MAX_DISTANCE);

void forward(int speedLeft = MOTOR_SPEED, int speedRight = MOTOR_SPEED) {
    analogWrite(ENA, speedRight);
    analogWrite(ENB, speedLeft);
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
}

void reverse(int speedLeft = MOTOR_SPEED, int speedRight = MOTOR_SPEED) {
    analogWrite(ENA, speedRight);
    analogWrite(ENB, speedLeft);
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
}

void stopMotors() {
    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
}

void setup() {
    // IR Sensor
    pinMode(IR_FRONT, INPUT);

    // Ultrasonic
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);

    // Motors
    pinMode(ENA, OUTPUT);
```

```
pinMode(IN1, OUTPUT);
pinMode(IN2, OUTPUT);
pinMode(ENB, OUTPUT);
pinMode(IN3, OUTPUT);
pinMode(IN4, OUTPUT);

Serial.begin(9600);
}

void loop() {
  int irFront = digitalRead(IR_FRONT);
  int distance = sonarFront.ping_cm();

  Serial.print("IR: ");
  Serial.print(irFront);
  Serial.print(" | Distance: ");
  Serial.print(distance);
  Serial.print(" cm | ");

  // IR detects object (LOW)
  if (irFront == LOW && distance >= 0) {
    if (distance <= TOO_CLOSE) {
      // Too close - reverse
      Serial.println("REVERSE");
      reverse(100, 100);
    } else if (distance <= FOLLOW_DISTANCE) {
      // Within follow range - move forward
      Serial.println("FORWARD");
      forward(120, 120);
    } else {
      // Object too far - stop
      Serial.println("STOP");
      stopMotors();
    }
  } else {
    // No object detected - stop
    Serial.println("STOP");
    stopMotors();
  }
}
```

3. Implementation

3.1 Build and Assembly

1. Assembling the Chassis

Robot assembly began with assembling the 2WD (two-wheel drive) chassis. Dual shaft BO motors were attached to the frame using screws and brackets. The trolley wheel was mounted at the front to support balance and movement.

2. Mounting the Wheels

The 65mm BO Motor Wheels were attached to the motor shafts. Motor wires, which were soldered to the BO Motors, were carefully routed to avoid tangling during motion.

3. Connecting the Motor Driver (L298N)

Mount the Arduino UNO module and the Motor Driver (L298N) module on the top of the Chassis using a glue gun. Connect the DC motors to the L298N motor driver module using their respective motor wires. These motor wires were connected in such a way that each pair of motors from a particular side (left or right) shared the same control signals, allowing us to use a single motor driver. After that, the Power Input Pins (IN1, IN2, IN3, IN4) and Control Pins (ENA, ENB) were connected to the Arduino UNO.

4. Attaching the Ultrasonic Sensor and IR Sensor

The IR sensor and the HC-SR04 ultrasonic sensor were mounted together at the very front of the robot, positioned to face directly forward. This co-location ensures that both sensors are detecting the same object, which is critical for the logic to work correctly. These sensors were then connected to specific digital pins on the Arduino UNO.

5. Power Supply

A 2-cell 18650 battery holder was fixed to the underside of the chassis using a glue gun. An ON/OFF switch was installed in-line with the power connection to control the robot's operation. The system was powered using two Hongli 2000mAh 3.7V 18650 Li-Ion Cell Batteries, which offered sufficient voltage and current for both the motors and control circuitry.

6. Breadboard and Wiring

A mini breadboard, placed at the top of the chassis, was used to organize the power connections, especially for distributing VCC and GND lines to various components. Jumper wires were used to interconnect all the modules, including the Arduino and Motor Driver. To maintain structural cleanliness and prevent loose wiring from interfering with movement, cable ties were used for proper cable management.

7. Uploading the Code

The Arduino was connected to a computer using a USB cable, and the final code was uploaded to the Arduino using the Arduino IDE. After a final check of all connections, the robot was ready for testing.

3.2 Challenges faced and how they were resolved

During the building and testing phase, we encountered several practical challenges that required careful problem-solving. The issues we faced, and the solutions implemented are as follows:

1. IR False Positives

- i. Observation: Ambient light interference caused erroneous detections.
- ii. Analysis: Uncontrolled lighting conditions triggered the IR sensor prematurely, leading

to false positives.

- iii. Mitigation: Adjusted the potentiometer to limit the detection range to 2–20 cm and conducted tests in controlled indoor lighting to minimize interference.

4. Testing and Results

4.1 Tests Performed

To evaluate the performance of the Object Tracking Robot, a series of tests were conducted in a controlled indoor environment on a flat area. The tests focused on verifying sensor accuracy, tracking stability, and system reliability.

1. Sensor Validation

Recorded IR and ultrasonic distance data via Serial for objects at distances ranging from 5-50 cm.

2. Static Following

Fixed an object at varying distances and confirmed the robot's forward, reverse, or stop responses.

3. Dynamic Engagement

Moved an object toward or away from the robot to assess pursuit and reaction times.

4. Proximity Safety

Approached an object to within <7 cm to verify the reverse action.

5. Loss Recovery

Removed the object to check for a stop response, then reintroduced it to evaluate re-engagement.

4.2 Observations and Outcomes

1. The IR sensor consistently triggered below 18 cm, while the ultrasonic sensor provided distance measurements with an accuracy of ± 1 cm, confirming reliable sensor performance across the tested range during the Sensor Validation test.
2. In the Static Following test, the robot maintained an average distance of 10 cm within the follow range (8-17 cm) when the object was stationary, with reverse action activating below 7 cm and stopping beyond 17 cm as expected.
3. During the Proximity Safety test, the robot reliably reversed when approaching within <7 cm, preventing collisions in most trials.
4. The Loss Recovery test showed that the robot stopped immediately upon object

removal and resumed tracking upon reintroduction, demonstrating effective re-engagement. Outcomes indicated an overall success rate of 94% across all tests, with the system proving robust in controlled conditions. Serial logs supported consistent behaviour, though rapid object movements highlighted a need for improved response tuning.

5. Conclusion and Future Scope

This project successfully demonstrates the design and implementation of a robust, reactive object-tracking robot, transitioning from simple obstacle avoidance to active, continuous target pursuit. By effectively integrating an IR sensor for initial object detection with an ultrasonic sensor for precise distance ranging, the robot made real-time decisions based on a threshold-based control model. It reliably maintained a set following distance, reversed to avoid collisions, and stopped when the target was lost. This implementation validates the core principles of basic sensor fusion and reactive control, while clearly identifying the inherent limitations of IR sensors concerning ambient light and target surface reflectivity. The result is a cost-effective and responsive prototype that serves as an excellent foundation for understanding interactive robotics.

Looking ahead, several upgrades can significantly expand the robot's capabilities. The most immediate improvement would be to replace the threshold-based logic with a PID controller. This would eliminate the abrupt forward and reverse motions and allow the robot to modulate its speed proportionally to the error, resulting in much smoother and more stable tracking. To overcome the narrow, forward-facing detection, the sensor suite could be mounted on a servo motor to actively scan for and lock onto the target. For more advanced tracking, replacing the IR sensor with a vision system would enable sophisticated object tracking based on color or shape. These improvements would transform the current prototype into a more intelligent and adaptable system, with expanded applications in logistics, assistive robotics, and automated surveillance.

As a to-do project for future extension, the robot could be enhanced into an ML/DL-based human tracking system by migrating the control platform to a more powerful single-board computer such as the NVIDIA Jetson Nano or Raspberry Pi 5. This upgrade would involve integrating a USB or CSI camera module for real-time video capture, leveraging pre-trained deep learning models (e.g., YOLO for human detection and DeepSORT for multi-object tracking) via frameworks like TensorFlow or PyTorch. The system could then perform advanced tasks such as identifying and following specific individuals based on facial recognition or pose estimation, while incorporating the existing ultrasonic sensor for collision avoidance. This ML/DL integration would enable operation in dynamic, unstructured environments (e.g., crowded spaces), with potential for cloud-based model fine-tuning and edge AI acceleration on the Jetson for low-latency inference. Implementation steps would include hardware interfacing via GPIO, model training on a dataset of human trajectories, and ROS (Robot Operating System) for modular software architecture, paving the way for applications in personal assistance, security patrolling, or eldercare companionship.

6. GitHub Repository

<https://github.com/aqib-m31/Object-Tracking-Robot>